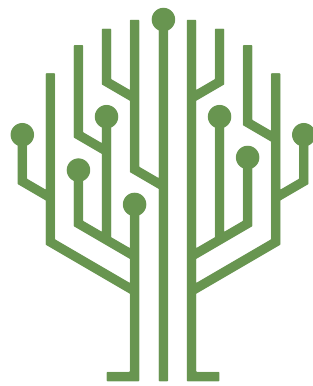




Green Pace

Green Pace Secure Development Policy

Contents	1
Overview	2
Purpose	2
Scope	2
Module Three Milestone	2
Ten Core Security Principles	2
C/C++ Ten Coding Standards	3
Coding Standard 1	4
Coding Standard 2	5
Coding Standard 3	6
Coding Standard 4	7
Coding Standard 5	8
Coding Standard 6	9
Coding Standard 7	10
Coding Standard 8	11
Coding Standard 9	13
Coding Standard 10	14
Defense-in-Depth Illustration	15
Project One	15
1. Revise the C/C++ Standards	15
2. Risk Assessment	15
3. Automated Detection	15
4. Automation	15
5. Summary of Risk Assessments	16
6. Create Policies for Encryption and Triple A	16
7. Map the Principles	17
Audit Controls and Management	18
Enforcement	18
Exceptions Process	18
Distribution	19
Policy Change Control	19
Policy Version History	19
Appendix A Lookups	19
Approved C/C++ Language Acronyms	19

Overview

Software development at Green Pace requires consistent implementation of secure principles to all developed applications. Consistent approaches and methodologies must be maintained through all policies that are uniformly defined, implemented, governed, and maintained over time.

Purpose

This policy defines the core security principles; C/C++ coding standards; authorization, authentication, and auditing standards; and data encryption standards. This article explains the differences between policy, standards, principles, and practices (guidelines and procedure): [Understanding the Hierarchy of Principles, Policies, Standards, Procedures, and Guidelines](#).

Scope

This document applies to all staff that create, deploy, or support custom software at Green Pace.

Module Three Milestone

Ten Core Security Principles

Principles	Write a short paragraph explaining each of the 10 principles of security.
1. Validate Input Data	Anytime a user enters data that the application plans to use, it should be validated first. If users enter bad data, it could result in a system crash or even a security attack. Due to this risk, the system should verify the data is correctly formatted and safe before use.
2. Heed Compiler Warnings	You should never ignore compiler warnings since they alert you of potential errors in the code. A compiler warning could indicate a security vulnerability or a code bug/vulnerability. You should always fix these warnings to avoid issues and ensure the code is secure.
3. Architect and Design for Security Policies	The security of an application should be considered at the beginning and throughout development. Designing an application with security in mind will help improve overall security and reduce the risk of vulnerabilities. For example, developers should plan for how sensitive data will be secured in storage and during transit.
4. Keep It Simple	Keeping code simple improves its maintainability, security, and readability. The more complex a system is, the more likely there will be vulnerabilities and bugs present. This is due to complex systems being more difficult to read and test.
5. Default Deny	By default, access should be denied, and only specify access should be granted. This is the safest approach since users have to first request permission, reducing the likelihood of unauthorized access happening or a leak of sensitive data.
6. Adhere to the Principle of Least Privilege	You should only allow the least privilege necessary for a user, application, or process to perform its tasks. If an account is compromised by a malicious user or malware, the principle of least privilege helps to minimize the damage.
7. Sanitize Data Sent to Other Systems	Data should be cleaned and validated before sending it to another system, application, or database. This helps to prevent malformed data or malicious code from being processed by the receiver, improving both the security and reliability of the system.



Principles	Write a short paragraph explaining each of the 10 principles of security.
8. Practice Defense in Depth	Defense in depth means having multiple overlapping layers of security rather than relying on a single security tool or mechanism. This design ensures that if one layer of security fails, the other will still protect the system. For example, an application could use a combination of authorization, authentication, encryption, and firewall security.
9. Use Effective Quality Assurance Techniques	Effective and complete testing is important to ensuring an application is secure. Quality assurance techniques like scanning for vulnerabilities and automated testing can help to identify issues in the code before production releases. Finding these issues early and fixing them improves the quality of the code and improves the overall security before releasing the software to the public.
10. Adopt a Secure Coding Standard	A secure coding standard provides developers with best coding practices and guidelines to follow during development. Following these standards helps us to prevent introducing common vulnerabilities and encourages consistent development practices across projects, providing more maintainable code over time.

C/C++ Ten Coding Standards

Complete the coding standards portion of the template according to the Module Three milestone requirements. In Project One, follow the instructions to add a layer of security to the existing coding standards. Please start each standard on a new page, as they may take up more than one page. The first seven coding standards are labeled by category. The last three are blank so you may choose three additional standards. Be sure to label them by category and give them a sequential number for that category. Add compliant and noncompliant sections as needed to each coding standard.

Coding Standard 1

Coding Standard	Label	Data Type Validation
Data Type	STD-001-CPP	All input should be verified before use. Validating data types prevents runtime errors and unexpected behavior.

Noncompliant Code

This program accepts user input and immediately stores it in an integer variable without checking whether the input is actually a valid integer.

```
#include <iostream>

using namespace std;

int main() {
    int age;

    cin >> age;

    cout << "Age: " << age << endl;
}
```

Compliant Code

This program verifies that the user entered a valid integer before processing the value and displays an error message when invalid input is detected.

Compliant Code

```
#include <iostream>

using namespace std;

int main() {
    int age;

    if (cin >> age) {
        cout << "Age: " << age << endl;
    } else {
        cerr << "Invalid input." << endl;
    }
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Medium	High	2

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.17	invalidPointerCast	Detects unsafe type conversions and invalid casts.
SonarQube	10.5	cpp:S1905	Detects unnecessary or unsafe type casts.

Coding Standard 2

Coding Standard	Label	Data Value Validation
Data Value	STD-002-CPP	Even valid data types could contain invalid values. Checking the valid range ensures data meets business requirements.

Noncompliant Code

This program accepts a quantity value and processes it without confirming that it's within an acceptable range.

```
int quantity;
cin >> quantity;
processOrder(quantity);
```

Compliant Code

This program validates that the quantity is within the allowed range before processing the order.

```
int quantity;
cin >> quantity;
if (quantity >= 1 && quantity <= 100) {
    processOrder(quantity);
}
else {
    cerr << "Invalid quantity." << endl;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]



Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Low	Low	High	1

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.17	integerOverflow	Detects integer overflow and invalid numeric values.
SonarQube	10.5	cpp:S3519	Detects arithmetic operations that may overflow.

Coding Standard 3

Coding Standard	Label	String Correctness
String Correctness	STD-003-CPP	Using fixed-size character arrays can lead to buffer overflows while std::string manages memory safely.

Noncompliant Code

This program copies a string into a fixed-size character array without checking the length first which can cause a buffer overflow.

```
char username[10];

strcpy(username, "ThisStringIsWayTooSuperLong");
```

Compliant Code

This program uses std::string which automatically manages memory and prevents buffer overflow issues associated with fixed-size arrays.

```
#include <string>

std::string username = "ThisStringIsWayTooSuperLong";
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Likely	Medium	High	1

Automation



Tool	Version	Checker	Description Tool
Cppcheck	2.17	bufferAccessOutOfBounds	Detects unsafe string and buffer operations.
SonarQube	10.5	cpp:S578	Detects string manipulation that may lead to memory corruption.

Coding Standard 4

Coding Standard	Label	SQL Injection Prevention
SQL Injection	STD-004-CPP	Parameterized queries prevent malicious user input from being interpreted by the system as SQL commands.

Noncompliant Code

The SQL query is built by directly concatenating user input into the query string which makes the application vulnerable to SQL injection attacks.

```
string username;

cin >> username;

string query =

    "SELECT * FROM users WHERE username='"

    + username + "'";
```

Compliant Code

The SQL query uses parameterized statements to separate user input from SQL commands which prevents injection attacks.

```
sqlite3_stmt* stmt;

const char* query =

    "SELECT * FROM users WHERE username=?";

sqlite3_prepare_v2(db, query, -1, &stmt, nullptr);

sqlite3_bind_text(stmt, 1, username.c_str(), -1, SQLITE_STATIC);
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level



Severity	Likelihood	Remediation Cost	Priority	Level
Critical	Likely	Low	Critical	1

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.5	Security Hotspot - SQL Injection	Detects untrusted input used in SQL statements.
SonarQube	10.5	CWE-89	Identifies potential SQL injection vulnerabilities.

Coding Standard 5

Coding Standard	Label	Memory Protection
Memory Protection	STD-005-CPP	Checking valid bounds prevents invalid memory access and potential security vulnerabilities.

Noncompliant Code

This program writes to an array index outside the valid bounds of the array, which can cause memory corruption or the application to crash.

```
int numbers[5];

numbers[10] = 100;
```

Compliant Code

This program verifies that the index is within the valid range before accessing the container.

```
#include <vector>

vector<int> numbers(5);

if (index >= 0 &&
    index < numbers.size()) {
    numbers[index] = 100;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level



Severity	Likelihood	Remediation Cost	Priority	Level
Critical	Medium	High	Critical	1

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.17	memleak	Detects memory leaks.
Cppcheck	2.17	nullPointer	Detects null pointer dereferences.
SonarQube	10.5	cpp:S2259	Detects unsafe memory access and null dereferences.

Coding Standard 6

Coding Standard	Label	Assertions
Assertions	STD-006-CPP	Assertions could be disabled in production environments so critical validation should always be enforced at runtime.

Noncompliant Code

This program relies on an assertion to validate a pointer before use, but assertions could be disabled in production environments.

```
#include <cassert>

assert(user != nullptr);

user->login();
```

Compliant Code

This program performs an explicit runtime check and throws an exception when the pointer is invalid.

```
if (user == nullptr) {
    throw runtime_error(
        "User pointer is null");
}

user->login();
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Medium	Low	Medium	3



Automation

Tool	Version	Checker	Description Tool
SonarQube	10.5	cpp:S4274	Detects misuse of assertions and assumptions
Cppcheck	2.17	warning	Identifies assertions that may hide runtime failures.

Coding Standard 7

Coding Standard	Label	Exception Handling
Exceptions	STD-007-CPP	Ignoring exceptions hides errors and complicates the debugging process. Exceptions should be handled appropriately so they are visible and easy to debug.

Noncompliant Code

This program catches all exceptions but doesn't log, report, or handle them, which will make troubleshooting difficult.

```
try {
    processFile();
}
catch (...) {
}
```

Compliant Code

This program catches exceptions and outputs meaningful error logs to better assist with debugging and system monitoring.

```
try {
    processFile();
}
catch (const exception& e) {
    cerr << e.what() << endl;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]



Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Low	High	2

Automation

Tool	Version	Checker	Description Tool
SonarQube	10.5	cpp:S2738	Detects improper exception handling.
Cppcheck	2.17	exceptionHandling	Identifies uncaught or improperly handled exceptions.

Coding Standard 8

Coding Standard	Label	Resource Management
Resource Management	STD-008-CPP	Resources should be automatically managed whenever possible to avoid leaks and improve the reliability of the code.

Noncompliant Code

This program opens a file using a low-level file handle but doesn't ensure that the resource will be properly closed.

```
FILE* file =
    fopen("data.txt", "r");
fscanf(file, "%d", &value);
```

Compliant Code

This program uses C++ stream object which automatically release the resource when it goes out of scope.

```
#include <fstream>
ifstream file("data.txt");
if (file.is_open()) {
    file >> value;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Critical	Likely	Low	Critical	1



Automation

Tool	Version	Checker	Description Tool
SonarQube	10.5	Security Hotspot - Input Validation	Detects missing validation of user input.
Cppcheck	2.17	invalidFunctionArg	Identifies potentially unsafe external input usage.

Coding Standard 9

Coding Standard	Label	Integer Overflow Protection
Integer Overflow Protection	STD-009-CPP	Integer overflow can produce incorrect results and introduce security vulnerabilities. Checking valid bounds helps to prevent overflow from occurring.

Noncompliant Code

This program performs an addition that exceeds the maximum value an integer can store which will result in an overflow.

```
int total =
    INT_MAX + 1;
```

Compliant Code

This program checks whether the increment operation can be safely performed before incrementing the value.

```
if (value <=
    numeric_limits<int>::max() - increment) {
    value += increment;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Medium	Medium	Medium	3

Automation



Tool	Version	Checker	Description Tool
Cppcheck	2.17	resourceLeak	Detects unreleased files, handles, and resources.
SonarQube	10.5	cpp:S2093	Detects resources not properly closed or released.

Coding Standard 10

Coding Standard	Label	Secure Logging
Secure Logging	STD-010-CPP	Sensitive information such as passwords should never be logged because logs may be accessed by unauthorized individuals.

Noncompliant Code

This program writes sensitive information, such as the user's password on a login attempt, directly to a log or console output.

```
cout << "Password: "
      << password
      << endl;
```

Compliant Code

This program logs only non-sensitive information while protecting sensitive user data, such as the entered user password.

```
cout << "User login attempt."
      << endl;
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): [Name the principle and explain how it maps to this standard.]

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Medium	High	2

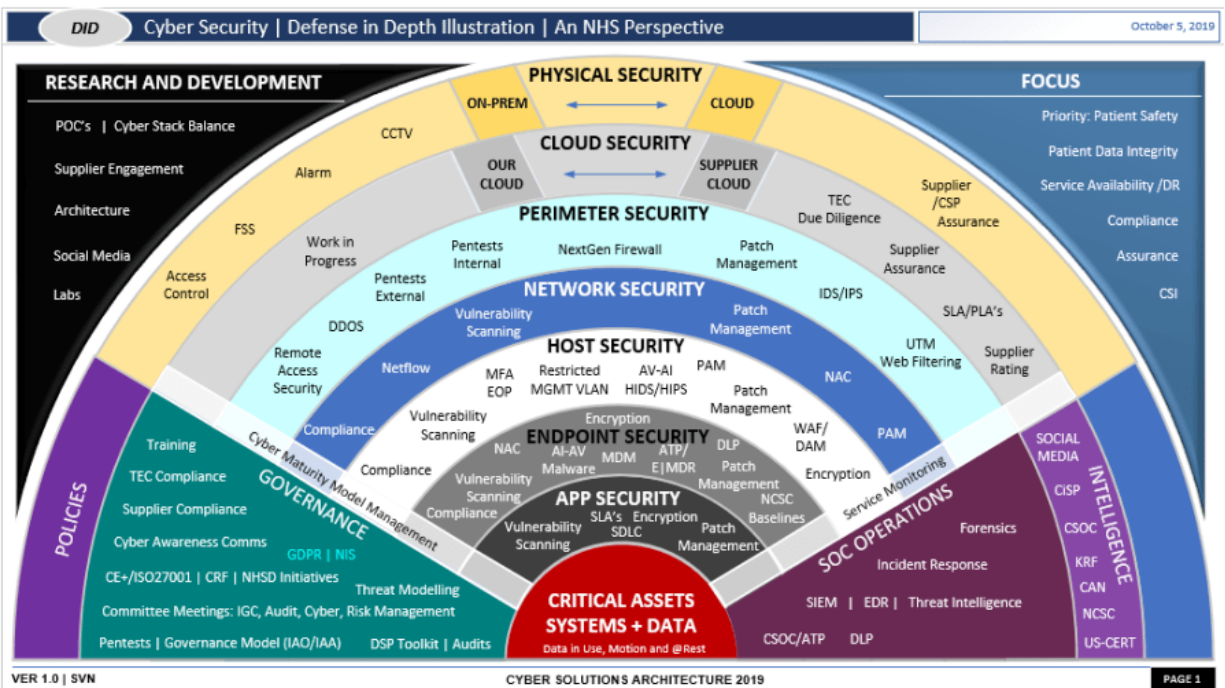
Automation



Tool	Version	Checker	Description Tool
Cppcheck	2.17	integerOverflow	Detects overflow and underflow conditions.
SonarQube	10.5	cpp:S3519	Detects unsafe integer arithmetic operations.

Defense-in-Depth Illustration

This illustration provides a visual representation of the defense-in-depth best practice of layered security.



Project One

There are seven steps outlined below that align with the elements you will be graded on in the accompanying rubric. When you complete these steps, you will have finished the security policy.

Revise the C/C++ Standards

You completed one of these tables for each of your standards in the Module Three milestone. In Project One, add revisions to improve the explanation and examples as needed. Add rows to accommodate additional examples of compliant and noncompliant code. Coding standards begin on the security policy.

Risk Assessment

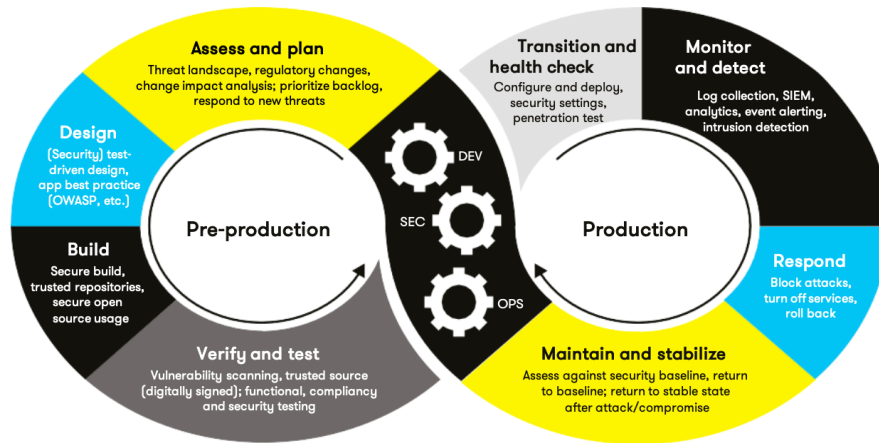
Complete this section on the coding standards tables. Enter high, medium, or low for each of the headers, then rate it overall using a scale from 1 to 5, 5 being the greatest threat. You will address each of the seven policy standards. Fill in the columns of severity, likelihood, remediation cost, priority, and level using the values provided in the appendix.

Automated Detection

Complete this section of each table on the coding standards to show the tools that may be used to detect issues. Provide the tool name, version, checker, and description. List one or more tools that can automatically detect this issue and its version number, name of the rule or check (preferably with link), and any relevant comments or description—if any. This table ties to a specific C++ coding standard.

Automation

Provide a written explanation using the image provided.



Automation will be used for the enforcement of and compliance to the standards defined in this policy. Green Pace already has a well-established DevOps process and infrastructure. Define guidance on where and how to modify the existing DevOps process to automate enforcement of the standards in this policy. Use the DevSecOps diagram and provide an explanation using that diagram as context.

[Insert your written explanations here.]

Summary of Risk Assessments

Consolidate all risk assessments into one table including both coding and systems standards, ordered by standard number.

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-001-CPP	High	Medium	Medium	High	2
STD-002-CPP	High	Likely	Low	High	1
STD-003-CPP	High	Likely	Medium	High	1
STD-004-CPP	Critical	Likely	Low	Critical	1
STD-005-CPP	Critical	Medium	High	Critical	1
STD-006-CPP	Medium	Medium	Low	Medium	3
STD-007-CPP	High	Medium	Low	High	2
STD-008-CPP	Critical	Likely	Low	Critical	1
STD-009-CPP	Medium	Medium	Medium	Medium	3
STD-010-CPP	High	Medium	Medium	High	2

Create Policies for Encryption and Triple A

Include all three types of encryption (in flight, at rest, and in use) and each of the three elements of the Triple-A framework using the tables provided.

- Explain each type of encryption, how it is used, and why and when the policy applies.

- b. Explain each type of Triple-A framework strategy, how it is used, and why and when the policy applies.

Write policies for each and explain what it is, how it should be applied in practice, and why it should be used.

a. Encryption	Explain what it is and how and why the policy applies.
Encryption at rest	Whether being stored in a database, backup, file system, or cloud storage, all sensitive data at rest should be encrypted with the AES-256 standard. The AES-256 encryption policy protects the sensitive data if the device or system is compromised.
Encryption in flight	While sensitive data is being transmitted, network communications must establish a connection using TLS 1.3 or higher to protect the data from being intercepted or altered before reaching the destination target.
Encryption in use	When sensitive data is being processed, the data has to be protected by policies such as least privilege, memory protection, and secure execution, which reduces exposure of the data.

b. Triple-A Framework*	Explain what it is and how and why the policy applies.
Authentication	Authentication is proving that the user or system is who they say they are. It's used here to verify the identity of users through multi-factor authentication and their credentials.
Authorization	Authorization is proving that the already authenticated person or system has to request access before letting them proceed. This is applied by granting the least privilege necessary and role-based access to minimize protection points of vulnerabilities.
Accounting	Accounting is the record of events related to security for auditing. This can be done by collecting and retaining logs for compliance.

*Use this checklist for the Triple A to be sure you include these elements in your policy:

- User logins
- Changes to the database
- Addition of new users
- User level of access
- Files accessed by users

Map the Principles

Map the principles to each of the standards, and provide a justification for the connection between the two. In the Module Three milestone, you added definitions for each of the 10 principles provided. Now it's time to connect the standards to principles to show how they are supported by principles. You may have more than one principle for each standard, and the principles may be used more than once. Principles are numbered 1 through 10. You will list the number or numbers that apply to each standard, then explain how each of these principles supports the standard. This exercise demonstrates that you



have based your security policy on widely accepted principles. Linking principles to standards is a best practice.

NOTE: Green Pace has already successfully implemented the following:

- Operating system logs
- Firewall logs
- Anti-malware logs

The only item you must complete beyond this point is the Policy Version History table.

Audit Controls and Management

Every software development effort must be able to provide evidence of compliance for each software deployed into any Green Pace managed environment.

Evidence will include the following:

- Code compliance to standards
- Well-documented access-control strategies, with sampled evidence of compliance
- Well-documented data-control standards defining the expected security posture of data at rest, in flight, and in use
- Historical evidence of sustained practice (emails, logs, audits, meeting notes)

Enforcement

The office of the chief information security officer (OCISO) will enforce awareness and compliance of this policy, producing reports for the risk management committee (RMC) to review monthly. Every system deployed in any environment operated by Green Pace is expected to be in compliance with this policy at all times.

Staff members, consultants, or employees found in violation of this policy will be subject to disciplinary action, up to and including termination.

Exceptions Process

Any exception to the standards in this policy must be requested in writing with the following information:

- Business or technical rationale
- Risk impact analysis
- Risk mitigation analysis
- Plan to come into compliance
- Date for when the plan to come into compliance will be completed

Approval for any exception must be granted by chief information officer (CIO) and the chief information security officer (CISO) or their appointed delegates of officer level.

Exceptions will remain on file with the office of the CISO, which will administer and govern compliance.



Distribution

This policy is to be distributed to all Green Pace IT staff annually. All IT staff will need to certify acceptance and awareness of this policy annually.

Policy Change Control

This policy will be automatically reviewed annually, no later than 365 days from the last revision date. Further, it will be reviewed in response to regulatory or compliance changes, and on demand as determined by the OCISO.

Policy Version History

Version	Date	Description	Edited By	Approved By
1.0	08/05/2020	Initial Template	David Buksbaum	
2.0	05/31/2026	Coding Standards	Kimani Muhammad	
3.0	06/21/2026	Risk Assessment	Kimani Muhammad	

Appendix A Lookups

Approved C/C++ Language Acronyms

Language	Acronym
C++	CPP
C	CLG
Java	JAV